

✓ **Seasonal Agriculture Performance Analysis**

Major Data Analytics Project

Domain: Agriculture

Project Theme: Seasonal Agriculture Performance

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn

Project Goal

Investigate how agricultural performance varies across seasons and identify meaningful patterns, trends, relationships and differences in the available data.

1. Problem Statement

Agricultural activities are influenced by seasonal variations in environmental conditions, farming practices, resource availability and market conditions. As a result, agricultural performance may differ from one season to another. However, raw agricultural data does not clearly explain how agricultural performance changes across seasons or what patterns can be observed in different seasonal conditions. The problem is to analyze the given agricultural dataset and investigate seasonal differences in agricultural performance by identifying meaningful patterns, trends, relationships and variations within the available data.

2. Project Objectives

- Explore and understand the dataset.
- Clean and prepare the data for analysis.
- Examine how agricultural performance varies across seasons.
- Identify important seasonal patterns and trends.
- Investigate relationships between seasonal conditions and agricultural outcomes.
- Compare relevant groups within different seasons.
- Identify significant differences or unusual patterns.
- Apply appropriate statistical and visualization techniques.
- Interpret findings based on evidence from the dataset.
- Develop meaningful conclusions and data-driven recommendations.

✓ **3. Dataset**

The dataset contains farm-level agricultural records covering different seasons, locations, crops, environmental conditions, farming practices, production, costs, revenue, profit and resource usage.

Dataset file: [seasonal_agriculture_performance_dataset](#)

```
# Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")
```

#Load the dataset

```
file_path = "/content/seasonal_agriculture_performance_dataset.csv"
df = pd.read_csv(file_path)

print("Dataset loaded successfully.")
```

Dataset loaded successfully.

✓ 4. Initial Data Understanding

#Dataset shape

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

Number of rows: 4000
Number of columns: 28

Top 5 rows

```
df.head(5)
```

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_
0	SF10001	Andhra Pradesh	Rajkot	Wheat	Kharif	0.53	486.40	
1	SF10002	Maharashtra	Nalgonda	Maize	Kharif	6.53	855.40	
2	SF10003	Telangana	Warangal	Pulses	Rabi	4.86	455.60	
3	SF10004	Telangana	Indore	Rice	Kharif	4.50	753.20	
4	SF10005	Karnataka	Ludhiana	Maize	Zaid	4.21	101.60	

Bottom 5 rows

```
df.tail(5)
```

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg
3995	SF13996	Karnataka	Rajkot	Wheat	Kharif	4.72	865.40	
3996	SF13997	Karnataka	Krishna	Chilli	Kharif	8.77	947.50	
3997	SF13998	Andhra Pradesh	Erode	Pulses	Zaid	10.47	80.00	
3998	SF13999	Punjab	Warangal	Cotton	Kharif	1.73	475.00	
3999	SF14000	Punjab	Nashik	Wheat	Kharif	3.31	984.30	

```
# Random sample of records
```

```
df.sample(5, random_state=42)
```

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	A
555	SF10556	Andhra Pradesh	Nashik	Chilli	Rabi	7.94	456.80	
3491	SF13492	Punjab	Warangal	Wheat	Kharif	12.44	601.90	
527	SF10528	Maharashtra	Indore	Cotton	Kharif	6.35	1,213.70	
3925	SF13926	Madhya Pradesh	Nashik	Wheat	Rabi	12.52	374.00	
2989	SF12990	Gujarat	Rajkot	Chilli	Rabi	13.98	197.80	

```
# Column names
```

```
print(df.columns.tolist())
```

```
['Farm_ID', 'State', 'District', 'Crop', 'Season', 'Farm_Area_Hectares', 'Rainfall_mm',
```

```
# Data types and non-null counts
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4000 entries, 0 to 3999
```

```
Data columns (total 28 columns):
```

#	Column	Non-Null Count	Dtype
0	Farm_ID	4000 non-null	object
1	State	4000 non-null	object
2	District	4000 non-null	object
3	Crop	4000 non-null	object
4	Season	4000 non-null	object
5	Farm_Area_Hectares	4000 non-null	float64
6	Rainfall_mm	3952 non-null	float64
7	Avg_Temperature_C	4000 non-null	float64
8	Humidity_pct	4000 non-null	float64
9	Sunlight_Hours_Day	4000 non-null	float64
10	Soil_pH	4000 non-null	float64
11	Soil_Moisture_pct	3960 non-null	float64

```
12 Nitrogen_kg_ha          4000 non-null float64
13 Phosphorus_kg_ha        4000 non-null float64
14 Potassium_kg_ha         4000 non-null float64
15 Irrigation_Method        4000 non-null object
16 Fertilizer_kg_ha         4000 non-null float64
17 Pesticide_Litre_ha       4000 non-null float64
18 Seed_Quality_Score       4000 non-null float64
19 Yield_Tonnes_Ha          3968 non-null float64
20 Production_Tonnes        4000 non-null float64
21 Market_Price_INR_Tonne   4000 non-null int64
22 Total_Cost_INR           4000 non-null int64
23 Revenue_INR              4000 non-null int64
24 Profit_INR               4000 non-null int64
25 Water_Used_m3            4000 non-null int64
26 Water_Efficiency_t_per_1000m3 4000 non-null float64
27 Disease_Pest_Risk_pct    4000 non-null float64
dtypes: float64(17), int64(5), object(6)
memory usage: 875.1+ KB
```

```
# Numerical summary statistics

df.describe()
```

	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours
count	4,000.00	3,952.00	4,000.00	4,000.00	4,000.00
mean	7.89	601.15	26.82	63.21	10.14
std	4.22	288.93	3.82	11.96	1.81
min	0.50	80.00	16.20	25.00	0.00
25%	4.26	370.20	23.90	54.77	4.00
50%	7.89	582.00	26.90	63.00	7.00
75%	11.65	834.95	29.60	71.90	10.00
max	15.00	1,395.20	39.70	95.00	15.00

```
# Categorical summary

df.describe(include="object")
```

	Farm_ID	State	District	Crop	Season	Irrigation_Method
count	4000	4000	4000	4000	4000	4000
unique	4000	8	10	8	3	4
top	SF13984	Andhra Pradesh	Ludhiana	Rice	Kharif	Flood
freq	1	529	427	690	1779	1310

5. Data Quality Analysis

```
# Missing values
```

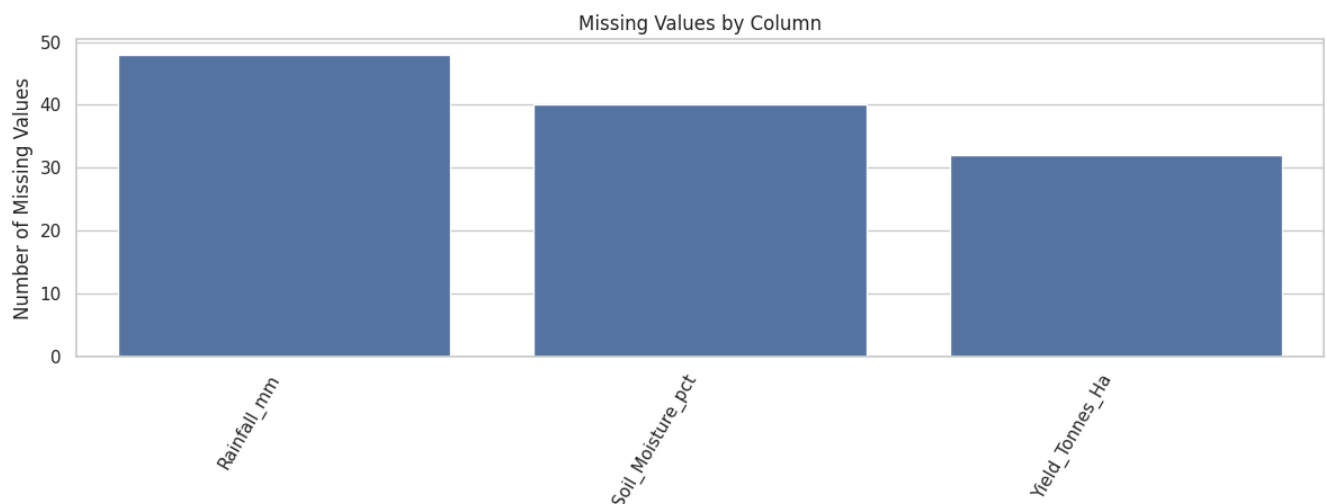
```
missing = df.isnull().sum().sort_values(ascending=False)
missing_percentage = (df.isnull().mean() * 100).sort_values(ascending=False)

missing_summary = pd.DataFrame({"Missing_Count": missing,
                                "Missing_Percentage": missing_percentage.round(2)})
missing_summary[missing_summary["Missing_Count"] > 0]
```

	Missing_Count	Missing_Percentage
Rainfall_mm	48	1.20
Soil_Moisture_pct	40	1.00
Yield_Tonnes_Ha	32	0.80

```
# Visualize missing values
```

```
plt.figure(figsize=(12, 5))
missing_plot = missing[missing > 0]
if len(missing_plot) > 0:
    sns.barplot(x=missing_plot.index, y=missing_plot.values)
    plt.xticks(rotation=60, ha="right")
    plt.ylabel("Number of Missing Values")
    plt.xlabel("Column")
    plt.title("Missing Values by Column")
    plt.tight_layout()
    plt.show()
else:
    print("No missing values found.")
```



```
# Duplicate records
```

```
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:", duplicate_count)
```

Number of duplicate rows: 0

```
# Drop duplicate rows
```

```
df = df.drop_duplicates().reset_index(drop=True)
```

```
# Confirm that they are gone  
print("Remaining rows:", df.shape)
```

```
Remaining rows: (4000, 28)
```

```
# Check data types again
```

```
df.dtypes
```

	0
Farm_ID	object
State	object
District	object
Crop	object
Season	object
Farm_Area_Hectares	float64
Rainfall_mm	float64
Avg_Temperature_C	float64
Humidity_pct	float64
Sunlight_Hours_Day	float64
Soil_pH	float64
Soil_Moisture_pct	float64
Nitrogen_kg_ha	float64
Phosphorus_kg_ha	float64
Potassium_kg_ha	float64
Irrigation_Method	object
Fertilizer_kg_ha	float64
Pesticide_Litre_ha	float64
Seed_Quality_Score	float64
Yield_Tonnes_Ha	float64
Production_Tonnes	float64
Market_Price_INR_Tonne	int64
Total_Cost_INR	int64
Revenue_INR	int64
Profit_INR	int64
Water_Used_m3	int64
Water_Efficiency_t_per_1000m3	float64
Disease_Pest_Risk_pct	float64

dtype: object

```
# Check unique values for categorical columns
```

```
categorical_cols = df.select_dtypes(include="object").columns
```

```
for col in categorical_cols:
    print(f"--- Unique values in '{col}' ({df[col].nunique()} unique values) ---")
    print(df[col].dropna().unique()[:20])
```

```

--- Unique values in 'Farm_ID' (4000 unique values) ---
['SF10001' 'SF10002' 'SF10003' 'SF10004' 'SF10005' 'SF10006' 'SF10007'
 'SF10008' 'SF10009' 'SF10010' 'SF10011' 'SF10012' 'SF10013' 'SF10014'
 'SF10015' 'SF10016' 'SF10017' 'SF10018' 'SF10019' 'SF10020']
--- Unique values in 'State' (8 unique values) ---
['Andhra Pradesh' 'Maharashtra' 'Telangana' 'Karnataka' 'Gujarat'
 'Tamil Nadu' 'Punjab' 'Madhya Pradesh']
--- Unique values in 'District' (10 unique values) ---
['Rajkot' 'Nalgonda' 'Warangal' 'Indore' 'Ludhiana' 'Raichur' 'Guntur'
 'Krishna' 'Nashik' 'Erode']
--- Unique values in 'Crop' (8 unique values) ---
['Wheat' 'Maize' 'Pulses' 'Rice' 'Cotton' 'Chilli' 'Groundnut' 'Sugarcane']
--- Unique values in 'Season' (3 unique values) ---
['Kharif' 'Rabi' 'Zaid']
--- Unique values in 'Irrigation_Method' (4 unique values) ---
['Drip' 'Flood' 'Rainfed' 'Sprinkler']

```

```
# Check missing values after duplicate removal
```

```
df.isnull().sum().sort_values(ascending=False).head(10)
```

	0
Rainfall_mm	48
Soil_Moisture_pct	40
Yield_Tonnes_Ha	32
Farm_ID	0
Crop	0
District	0
Farm_Area_Hectares	0
Season	0
Avg_Temperature_C	0
Humidity_pct	0

```
dtype: int64
```

```
# median imputation for numerical columns
```

```
numeric_columns = df.select_dtypes(include=np.number).columns
```

```
for col in numeric_columns:
```

```
    if df[col].isnull().sum() > 0:
```

```
        df[col] = df[col].fillna(df[col].median())
```

```
print("Remaining missing values:", df.isnull().sum().sum())
```

```
Remaining missing values: 0
```



```
# Numerical and categorical columns
```

```
numeric_columns = df.select_dtypes(include=np.number).columns.tolist()
categorical_columns = df.select_dtypes(include="object").columns.tolist()
print("Numerical columns:")
print(numeric_columns)

print("\nCategorical columns:")
print(categorical_columns)
```

Numerical columns:

['Farm_Area_Hectares', 'Rainfall_mm', 'Avg_Temperature_C', 'Humidity_pct', 'Sunlight_Hours']

Categorical columns:

['Farm_ID', 'State', 'District', 'Crop', 'Season', 'Irrigation_Method']

✓ 6. Statistical Analysis

```
# Descriptive statistics
```

```
stats = df[numeric_columns].describe().T
stats["range"] = stats["max"] - stats["min"]
stats["IQR"] = stats["75%"] - stats["25%"]
```

```
stats
```

	count	mean	std	min	25%
Farm_Area_Hectares	4,000.00	7.89	4.22	0.50	4.26
Rainfall_mm	4,000.00	600.92	287.19	80.00	371.58
Avg_Temperature_C	4,000.00	26.82	3.82	16.20	23.90
Humidity_pct	4,000.00	63.21	11.96	25.00	54.77
Sunlight_Hours_Day	4,000.00	7.32	1.12	3.50	6.60
Soil_pH	4,000.00	6.70	0.64	5.20	6.26
Soil_Moisture_pct	4,000.00	26.51	6.68	8.00	21.80
Nitrogen_kg_ha	4,000.00	120.25	31.22	40.00	98.50
Phosphorus_kg_ha	4,000.00	57.78	17.02	15.00	46.20
Potassium_kg_ha	4,000.00	104.76	27.74	30.00	86.20
Fertilizer_kg_ha	4,000.00	186.04	60.26	40.00	143.97
Pesticide_Litre_ha	4,000.00	5.07	1.98	0.50	3.69
Seed_Quality_Score	4,000.00	0.83	0.10	0.65	0.74
Yield_Tonnes_Ha	4,000.00	5.24	13.31	0.30	1.05
Production_Tonnes	4,000.00	43.25	126.88	0.15	5.02
Market_Price_INR_Tonne	4,000.00	44,714.17	30,266.16	2,594.00	21,429.25
Total_Cost_INR	4,000.00	526,303.58	294,419.14	26,826.00	274,577.50
Revenue_INR	4,000.00	637,860.05	636,974.71	6,107.00	207,517.25
Profit_INR	4,000.00	111,556.46	545,081.89	-1,019,223.00	-148,731.75
Water_Used_m3	4,000.00	6,045.57	5,572.94	128.00	2,268.75
Water_Efficiency_t_per_1000m3	4,000.00	5.39	8.84	0.14	1.66
Disease_Pest_Risk_pct	4,000.00	46.37	12.42	5.00	37.30

```
# Median and standard deviation

statistical_summary = pd.DataFrame({
    "Mean": df[numeric_columns].mean(),
    "Median": df[numeric_columns].median(),
    "Std_Dev": df[numeric_columns].std(),
    "Min": df[numeric_columns].min(),
    "Max": df[numeric_columns].max()
})

statistical_summary
```

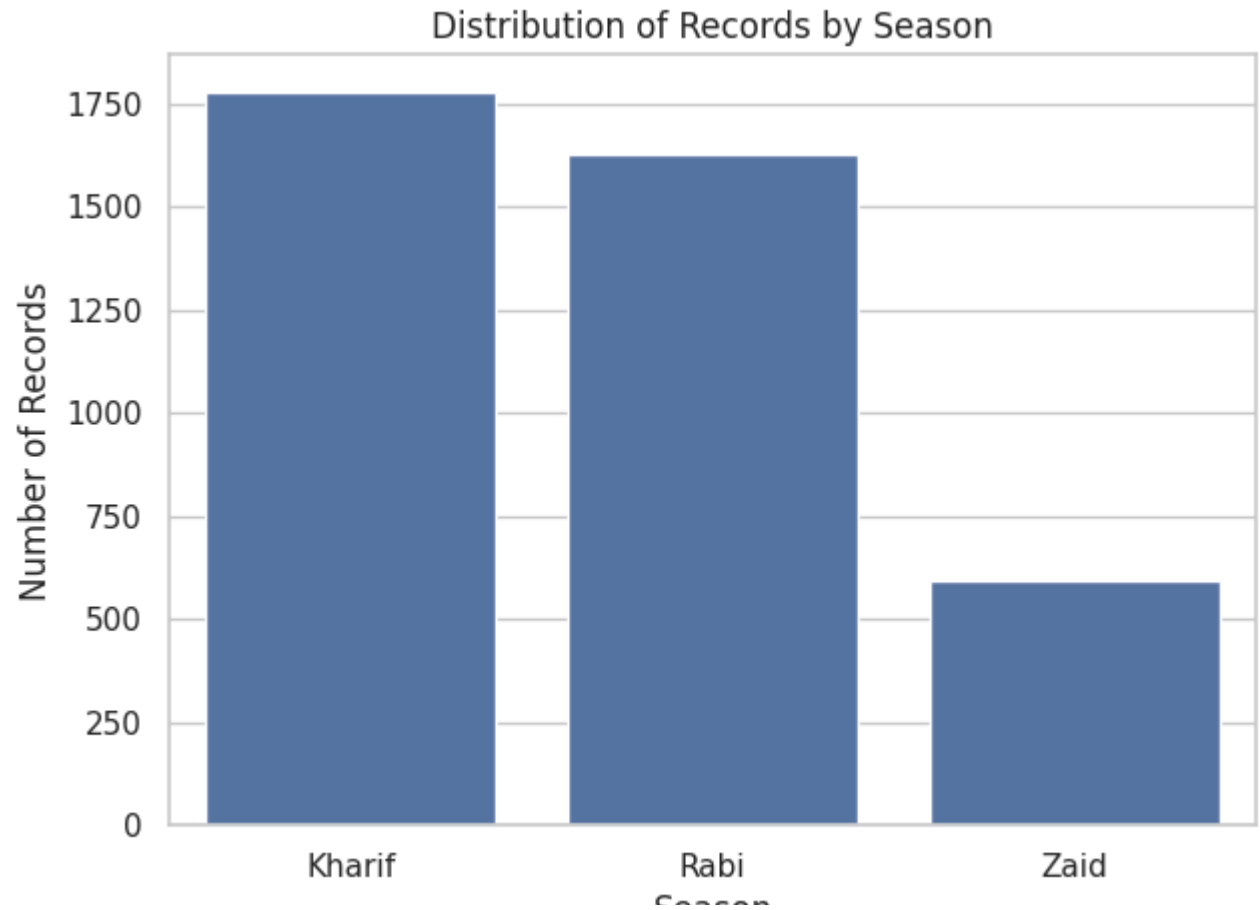
	Mean	Median	Std_Dev	Min	Max
Farm_Area_Hectares	7.89	7.89	4.22	0.50	15.00
Rainfall_mm	600.92	582.00	287.19	80.00	1,395.20
Avg_Temperature_C	26.82	26.90	3.82	16.20	39.70
Humidity_pct	63.21	63.00	11.96	25.00	95.00
Sunlight_Hours_Day	7.32	7.30	1.12	3.50	11.00
Soil_pH	6.70	6.69	0.64	5.20	8.40
Soil_Moisture_pct	26.51	26.40	6.68	8.00	47.00
Nitrogen_kg_ha	120.25	120.10	31.22	40.00	220.00
Phosphorus_kg_ha	57.78	57.50	17.02	15.00	120.00
Potassium_kg_ha	104.76	105.10	27.74	30.00	200.00
Fertilizer_kg_ha	186.04	186.25	60.26	40.00	400.00
Pesticide_Litre_ha	5.07	5.12	1.98	0.50	12.08
Seed_Quality_Score	0.83	0.83	0.10	0.65	1.00
Yield_Tonnes_Ha	5.24	1.74	13.31	0.30	101.44
Production_Tonnes	43.25	11.77	126.88	0.15	1,424.40
Market_Price_INR_Tonne	44,714.17	25,913.00	30,266.16	2,594.00	131,240.00
Total_Cost_INR	526,303.58	519,082.50	294,419.14	26,826.00	1,349,990.00
Revenue_INR	637,860.05	456,830.50	636,974.71	6,107.00	5,080,900.00
Profit_INR	111,556.46	3,673.00	545,081.89	-1,019,223.00	4,345,421.00
Water_Used_m3	6,045.57	4,435.00	5,572.94	128.00	40,902.00
Water_Efficiency_t_per_1000m3	5.39	2.94	8.84	0.14	79.80
Disease_Pest_Risk_pct	46.37	46.40	12.42	5.00	88.90

# Seasonal descriptive summary												
<pre> season_summary = df.groupby("Season")[numeric_columns].agg(["mean","median","std"]) season_summary </pre>												
	Farm_Area_Hectares			Rainfall_mm			Avg_Temperature_C			Humidity_pct		
	mean	median	std	mean	median	std	mean	median	std	mean	median	std
Season												
Kharif	7.94	8.18	4.28	849.20	853.10	183.69	28.45	28.50	2.51	71.81	71.90	8.5
Rabi	7.75	7.68	4.18	437.62	432.20	175.35	23.49	23.50	2.46	57.89	58.10	9.0
Zaid	8.09	8.09	4.16	304.65	288.20	157.24	31.04	31.00	2.55	52.01	52.35	9.0

--

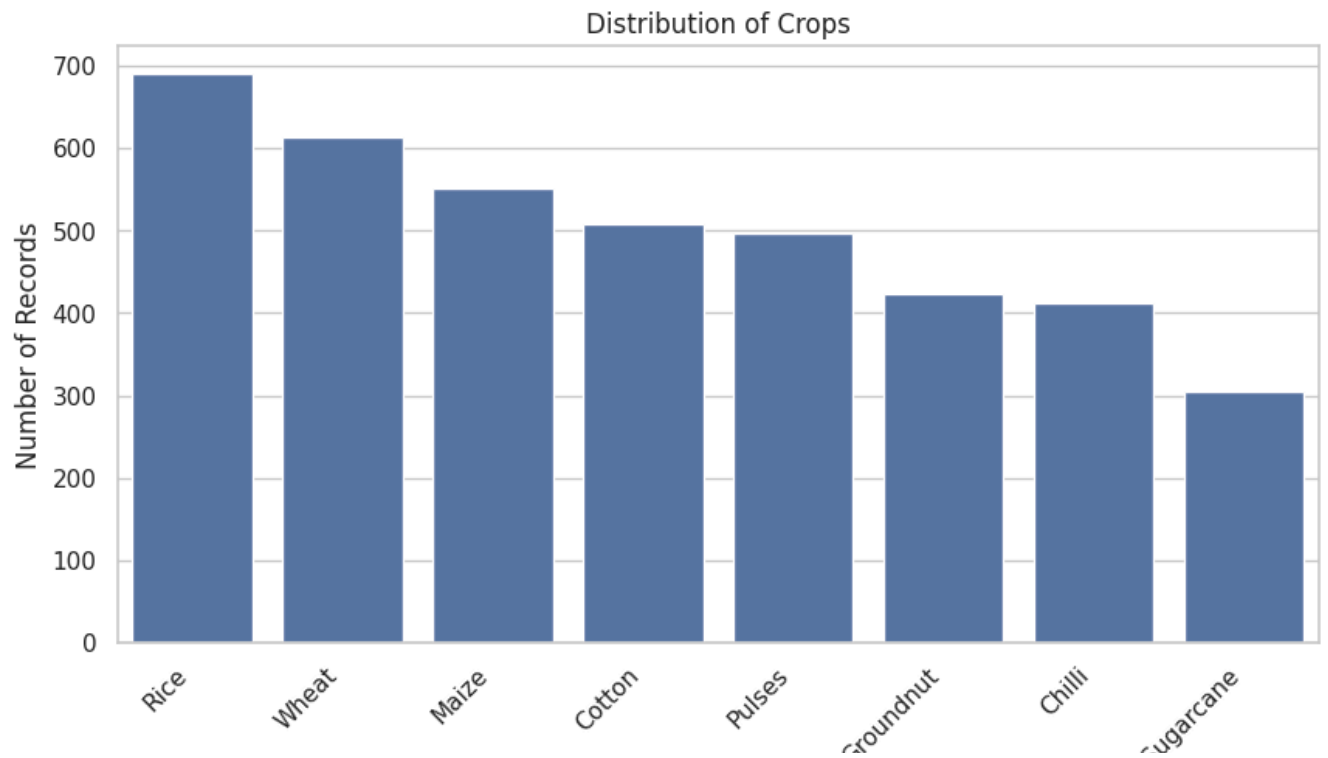
```
# Univariate: Season distribution
```

```
plt.figure(figsize=(7,5))
sns.countplot(data=df, x="Season")
plt.title("Distribution of Records by Season")
plt.xlabel("Season")
plt.ylabel("Number of Records")
plt.tight_layout
plt.show()
```



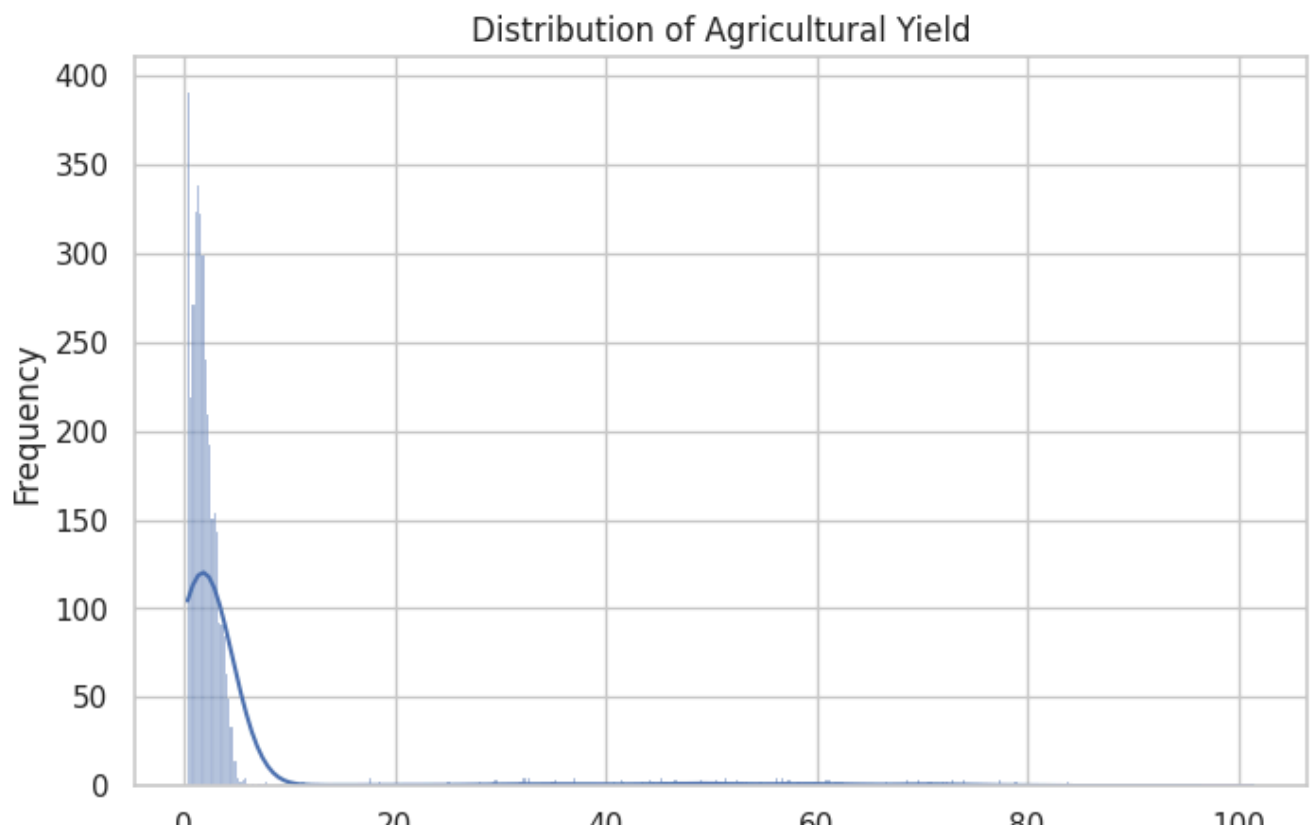
```
# Univariate: Crop distribution
```

```
plt.figure(figsize=(10,5))
sns.countplot(data=df, x="Crop", order=df["Crop"].value_counts().index)
plt.xticks(rotation=45, ha="right")
plt.title("Distribution of Crops")
plt.xlabel("Crop")
plt.ylabel("Number of Records")
plt.tight_layout
plt.show()
```



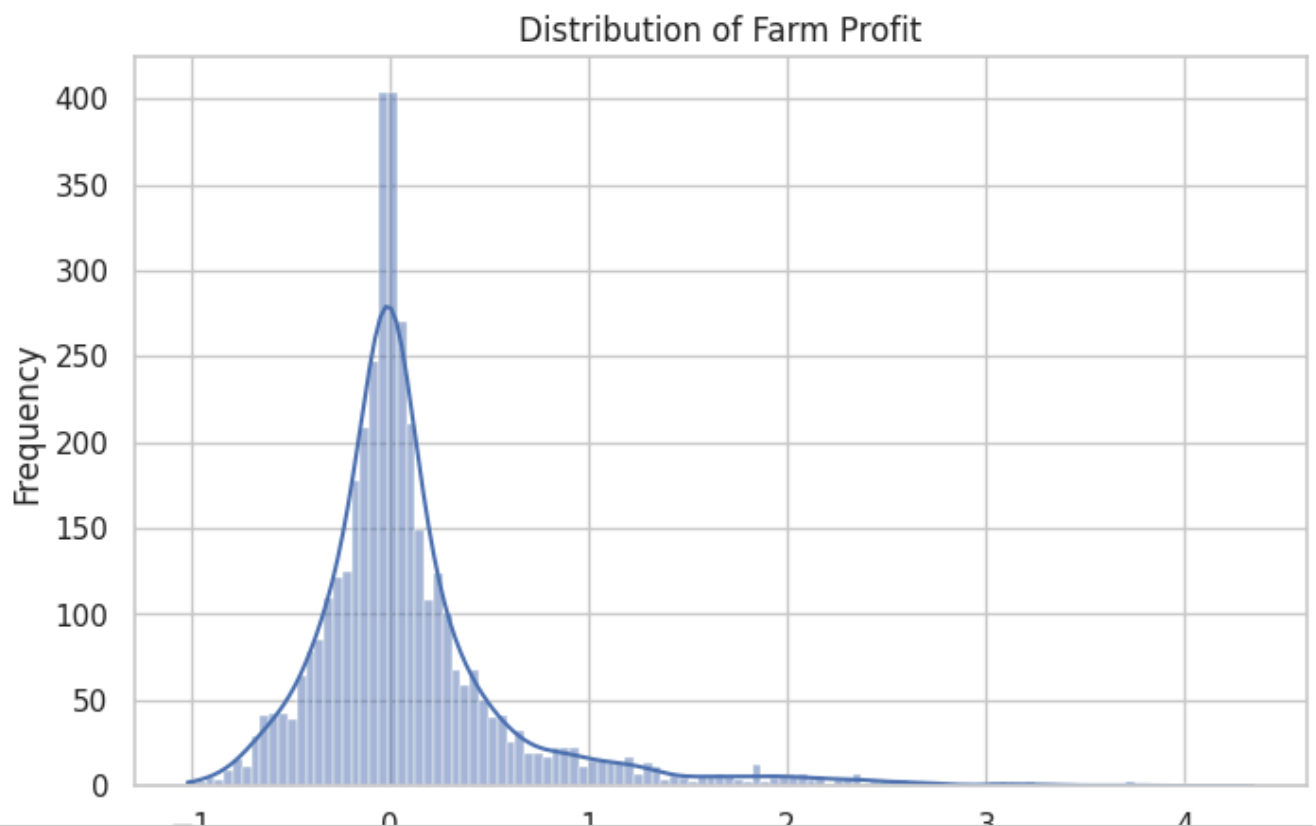
Univariate: Yield distribution

```
plt.figure(figsize=(8,5))
sns.histplot(data=df, x="Yield_Tonnes_Ha", kde=True)
plt.title("Distribution of Agricultural Yield")
plt.xlabel("Yield (tonnes/ha)")
plt.ylabel("Frequency")
plt.tight_layout
plt.show()
```



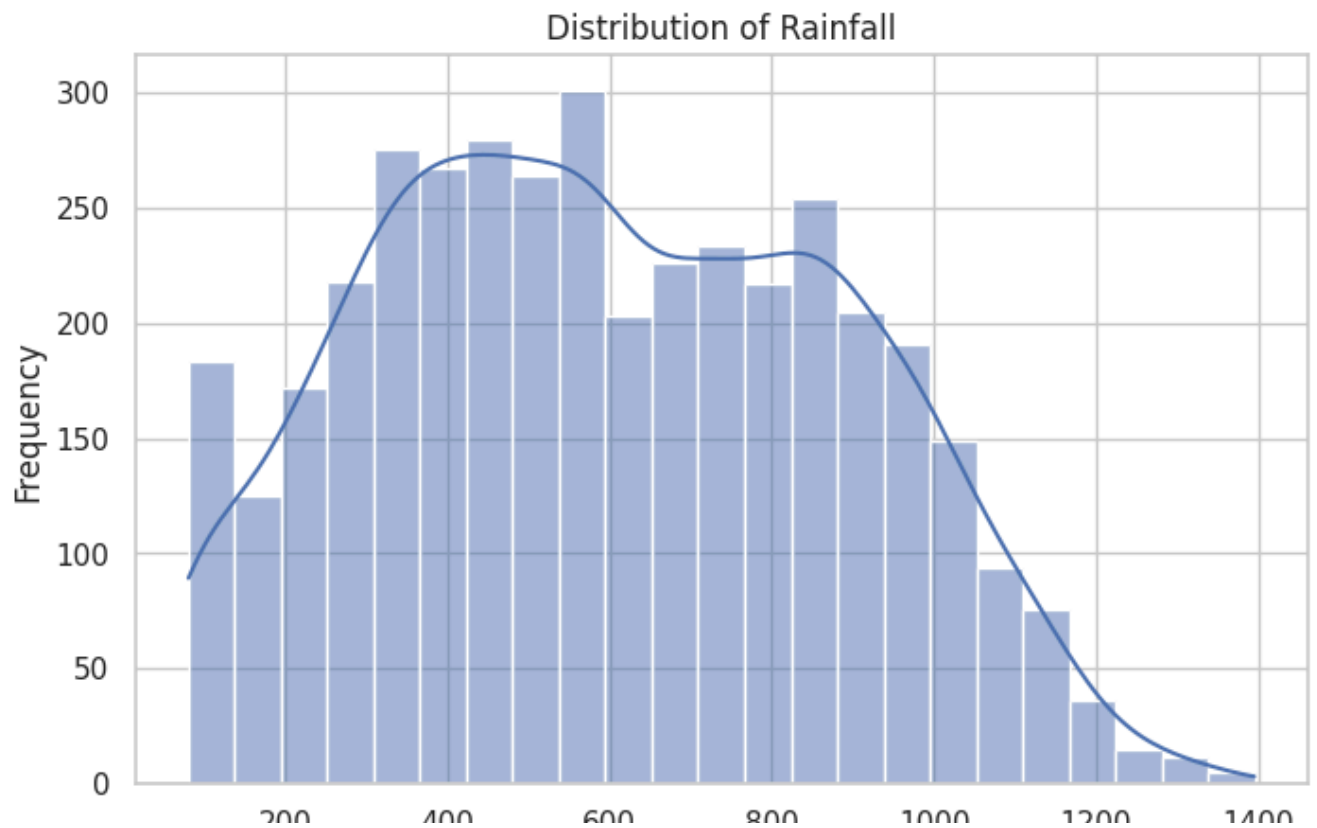
Univariate: Profit distribution

```
plt.figure(figsize=(8,5))
sns.histplot(data=df, x="Profit_INR", kde=True)
plt.title("Distribution of Farm Profit")
plt.xlabel("Profit (INR)")
plt.ylabel("Frequency")
plt.tight_layout
plt.show()
```



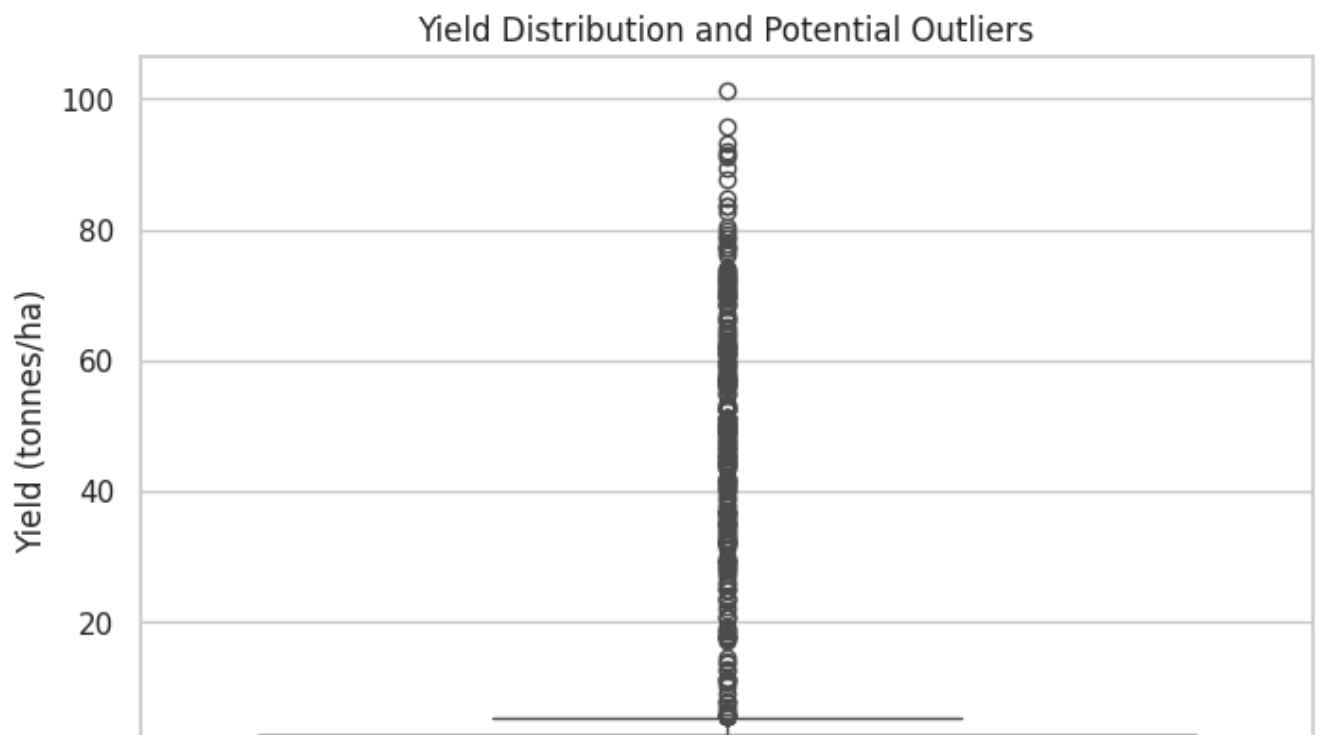
```
# Univariate: Rainfall distribution
```

```
plt.figure(figsize=(8,5))
sns.histplot(data=df, x="Rainfall_mm", kde=True)
plt.title("Distribution of Rainfall")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Frequency")
plt.tight_layout
plt.show()
```



```
# Univariate: Boxplot for Yield
```

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df, y="Yield_Tonnes_Ha")
plt.title("Yield Distribution and Potential Outliers")
plt.ylabel("Yield (tonnes/ha)")
plt.tight_layout
plt.show()
```

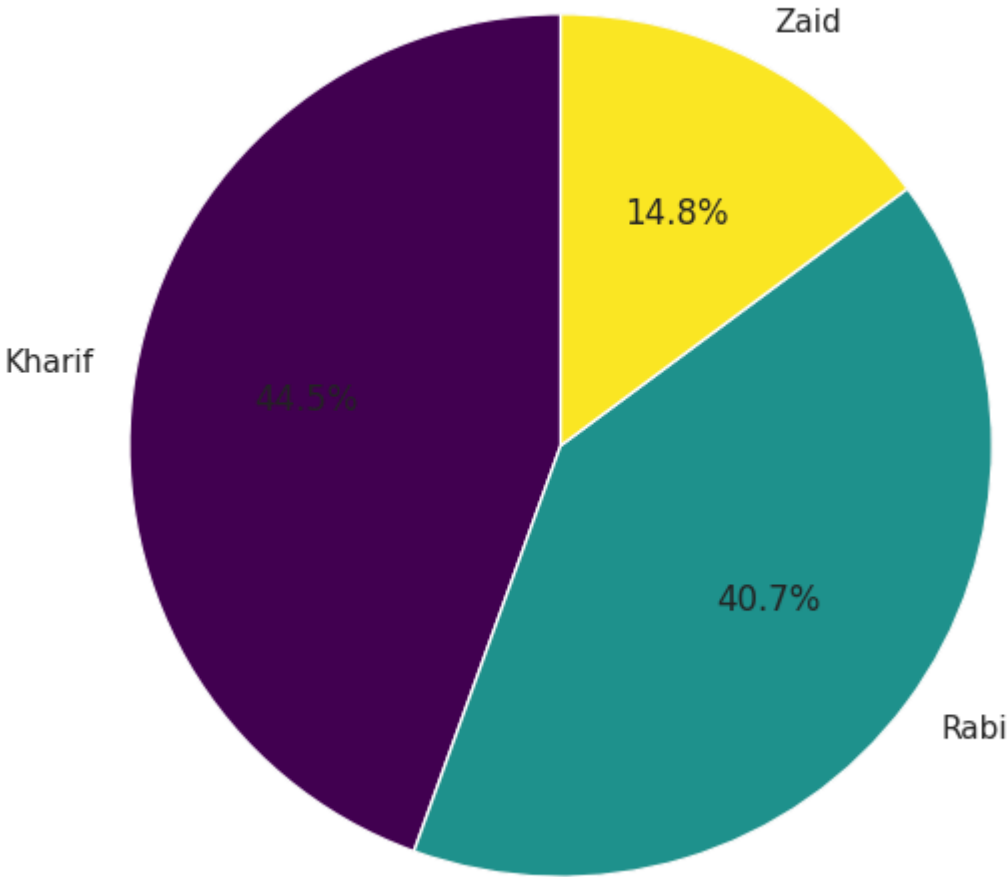


✓ 7. Outlier Analysis

Season distribution (Pie Chart)

```
plt.figure(figsize=(7,7))
df['Season'].value_counts().plot.pie(autopct='%1.1f%%', startangle=90, cmap='viridis')
plt.title("Distribution of Records by Season")
plt.ylabel('')
plt.tight_layout
plt.show()
```


Distribution of Records by Season



```
# IQR-based outlier summary for numerical columns
```

```
outlier_summary = []
```

```
for col in numeric_columns:
```

```
    q1 = df[col].quantile(0.25)
```

```
    q3 = df[col].quantile(0.75)
```

```
    iqr = q3 - q1
```

```
    lower = q1 - 1.5 * iqr
```

```
    upper = q3 + 1.5 * iqr
```

```
    count = ((df[col] < lower) | (df[col] > upper)).sum()
```

```
    outlier_summary.append({
```

```
        "Column": col,
```

```
        "Lower_Bound": lower,
```

```
        "Upper_Bound": upper,
```

```
        "Outlier_Count": count,
```

```
        "Outlier_Percentage": round(count / len(df) * 100, 2)
```

```
    })
```

```
outlier_summary = pd.DataFrame(outlier_summary).sort_values(
```

```
    "Outlier_Count", ascending=False
```

```
)
```

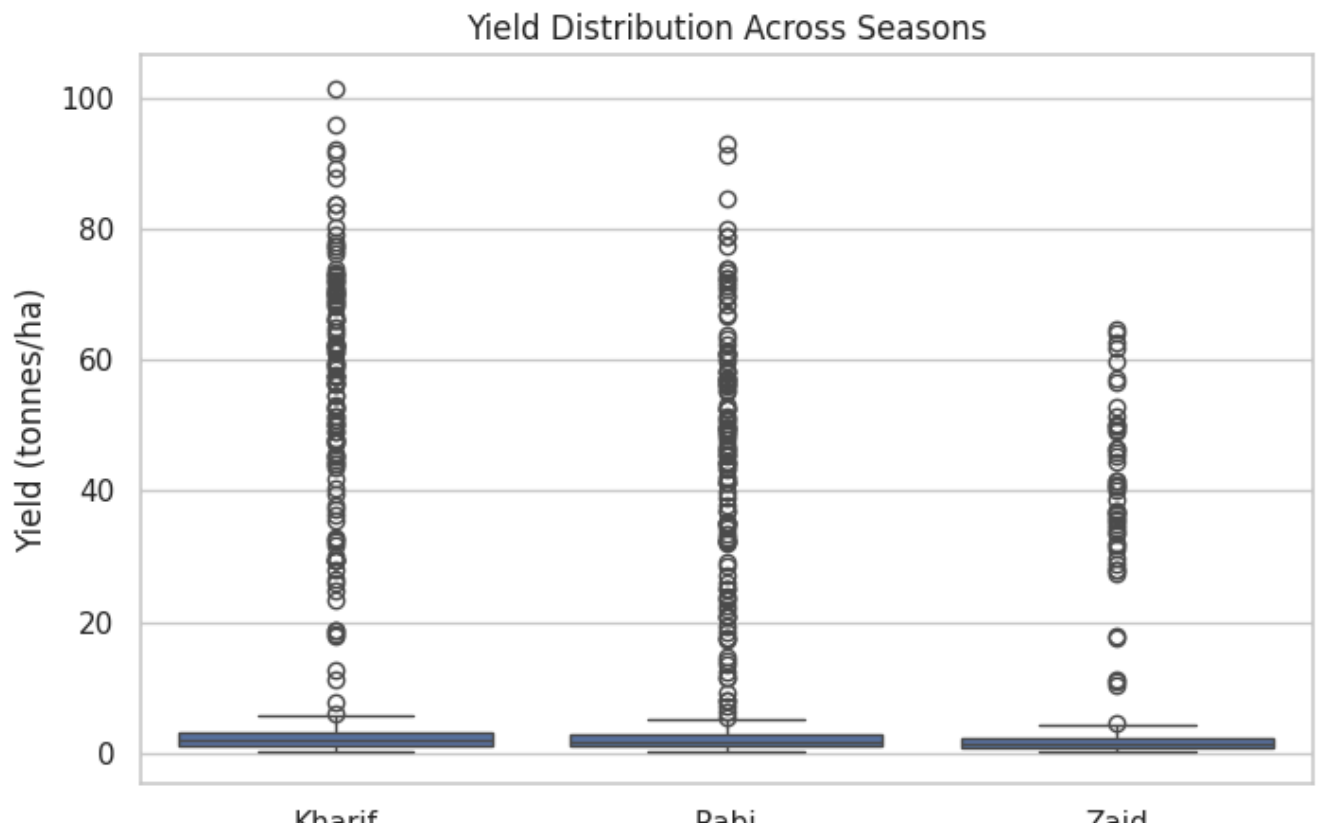
```
outlier_summary
```

	Column	Lower_Bound	Upper_Bound	Outlier_Count	Outlier_Percentage
18	Profit_INR	-696,111.00	763,567.00	404	0.0001
14	Production_Tonnes	-23.98	53.36	333	0.0001
20	Water_Efficiency_t_per_1000m3	-3.57	10.38	322	0.0001
13	Yield_Tonnes_Ha	-1.62	5.50	302	0.0001
19	Water_Used_m3	-6,072.75	16,171.25	276	0.0001
17	Revenue_INR	-736,519.00	1,780,911.00	240	0.0001
9	Potassium_kg_ha	30.10	179.70	32	0.0001
4	Sunlight_Hours_Day	4.35	10.35	26	0.0001
8	Phosphorus_kg_ha	11.40	104.20	16	0.0001
10	Fertilizer_kg_ha	19.14	352.04	15	0.0001
7	Nitrogen_kg_ha	34.41	205.31	10	0.0001
21	Disease_Pest_Risk_pct	10.90	81.30	10	0.0001
11	Pesticide_Litre_ha	-0.38	10.46	8	0.0001
3	Humidity_pct	29.09	97.59	6	0.0001
6	Soil_Moisture_pct	7.70	45.30	5	0.0001
2	Avg_Temperature_C	15.35	38.15	2	0.0001
0	Farm_Area_Hectares	-6.83	22.73	0	0.0001
1	Rainfall_mm	-319.02	1,522.57	0	0.0001
5	Soil_pH	4.94	8.46	0	0.0001
12	Seed_Quality_Score	0.48	1.17	0	0.0001
16	Total_Cost_INR	-456,551.00	1,493,125.00	0	0.0001
15	Market_Price_INR_Tonne	-48,845.38	138,553.62	0	0.0001

8. Bivariate Analysis

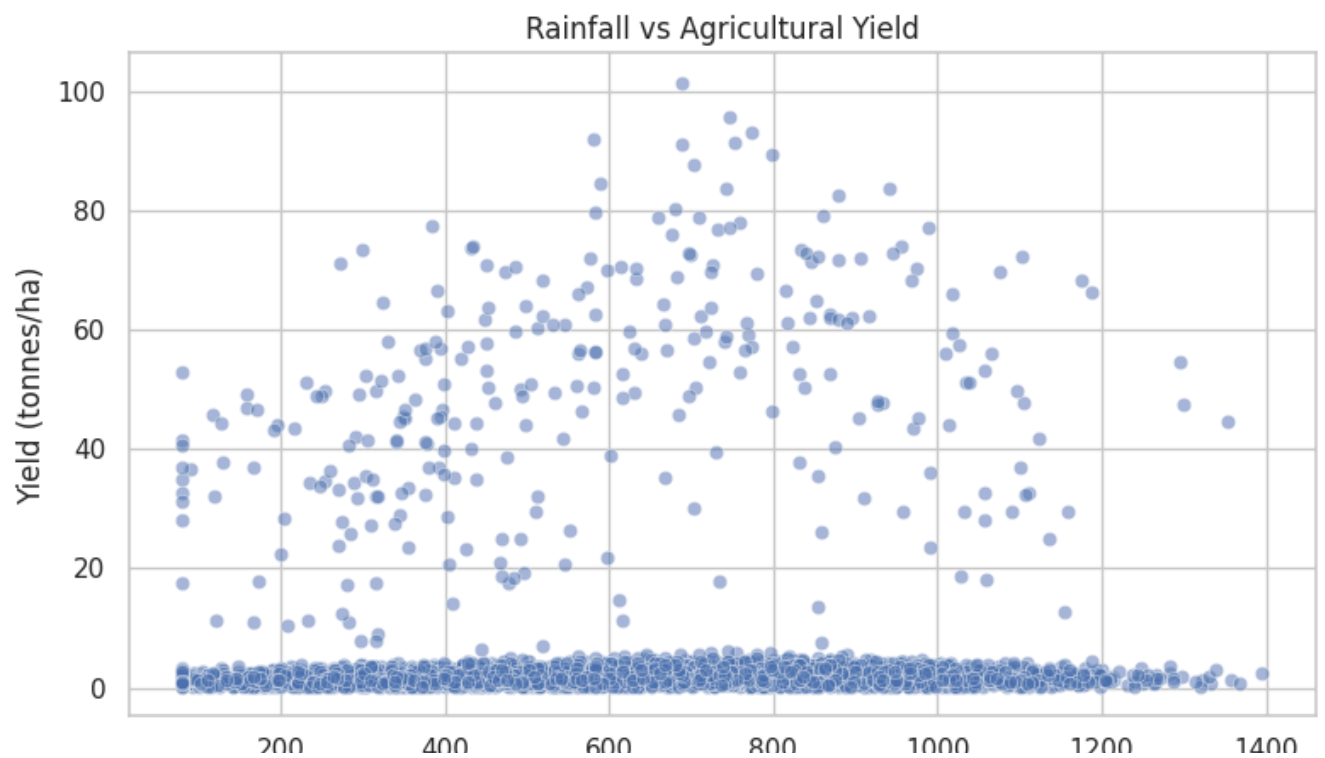
```
# Season vs Yield

plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="Season", y="Yield_Tonnes_Ha")
plt.title("Yield Distribution Across Seasons")
plt.xlabel("Season")
plt.ylabel("Yield (tonnes/ha)")
plt.tight_layout
plt.show()
```



Bivariate: Rainfall vs Yield

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x="Rainfall_mm", y="Yield_Tonnes_Ha", alpha=0.5)
plt.title("Rainfall vs Agricultural Yield")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Yield (tonnes/ha)")
plt.tight_layout()
plt.show()
```



9. Multivariate Analysis

```
#Multivariate: Pair Plot of Key Environmental and Performance Metrics by Season
selected_columns = ['Avg_Temperature_C', 'Rainfall_mm', 'Humidity_pct',
                    'Yield_Tonnes_Ha', 'Profit_INR', 'Season']
```

```
sns.pairplot(df[selected_columns], hue='Season',
              palette='viridis', plot_kws={'alpha': 0.6})
```

```
plt.suptitle('Pair Plot of Environmental and Performance Metrics by Season',
             y=1.02)
```

```
plt.tight_layout()
```

```
plt.show()
```

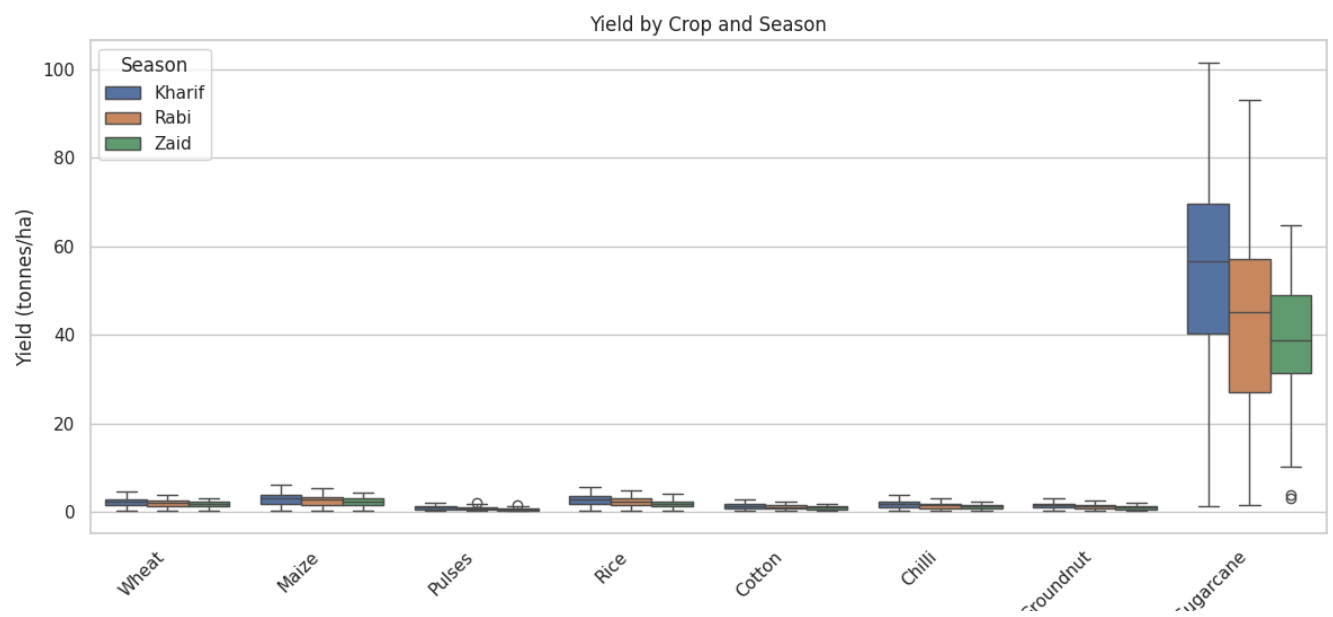


```
# Multivariate: Season, crop and yield
```

```
plt.figure(figsize=(12, 6))
```

```
sns.boxplot(data=df, x='Crop', y='Yield_Tonnes_Ha', hue='Season')
```

```
plt.xticks(rotation=45, ha='right')  
plt.title('Yield by Crop and Season')  
plt.xlabel('Crop')  
plt.ylabel('Yield (tonnes/ha)')  
plt.legend(title='Season')  
plt.tight_layout()  
plt.show()
```



10. Correlation Analysis

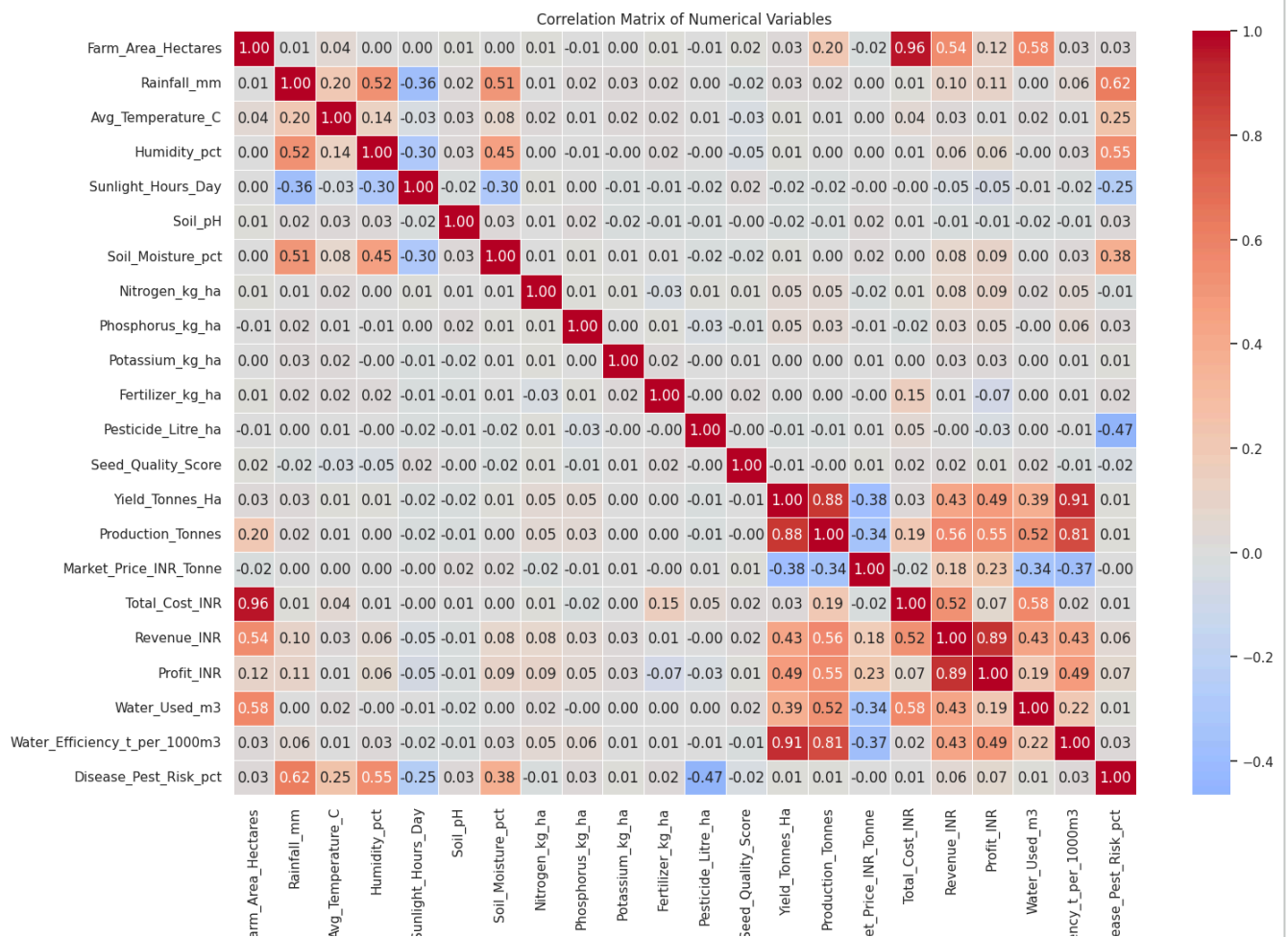
```
# Correlation heatmap for numerical variables
```

```
plt.figure(figsize=(16, 12))
```

```
corr = df.corr(numeric_only=True)
```

```
sns.heatmap(  
    corr,  
    annot=True,  
    fmt=".2f",  
    cmap="coolwarm",  
    center=0,  
    linewidths=0.5  
)
```

```
plt.title("Correlation Matrix of Numerical Variables")  
plt.tight_layout()  
plt.show()
```



```
# General seasonal summary
```

```
season_metrics = df.groupby("Season").agg(  
    Records=("Farm_ID", "count"),  
    Average_Yield=("Yield_Tonnes_Ha", "mean"),  
    Total_Production=("Production_Tonnes", "sum"),  
    Average_Profit=("Profit_INR", "mean"),  
    Total_Profit=("Profit_INR", "sum"),  
    Average_Water_Used=("Water_Used_m3", "mean")  
).round(2)
```

```
season_metrics
```

	Records	Average_Yield	Total_Production	Average_Profit	Total_Profit	Average
Season						
Kharif	1779	5.63	82,387.61	178,914.65	318289158	
Rabi	1627	5.04	67,498.88	87,689.47	142670768	
Zaid	594	4.64	23,098.35	-24,804.82	-14734066	

```
# Compare crop performance within each season
```

```
crop_season_summary = (  
    df.groupby(["Season", "Crop"])  
    .agg(  
        Average_Yield=("Yield_Tonnes_Ha", "mean"),  
        Average_Profit=("Profit_INR", "mean"),  
        Average_Water_Efficiency=("Water_Efficiency_t_per_1000m3", "mean")  
    )  
    .round(2)  
    .sort_values(["Season", "Average_Yield"], ascending=[True, False])  
)
```

```
crop_season_summary
```

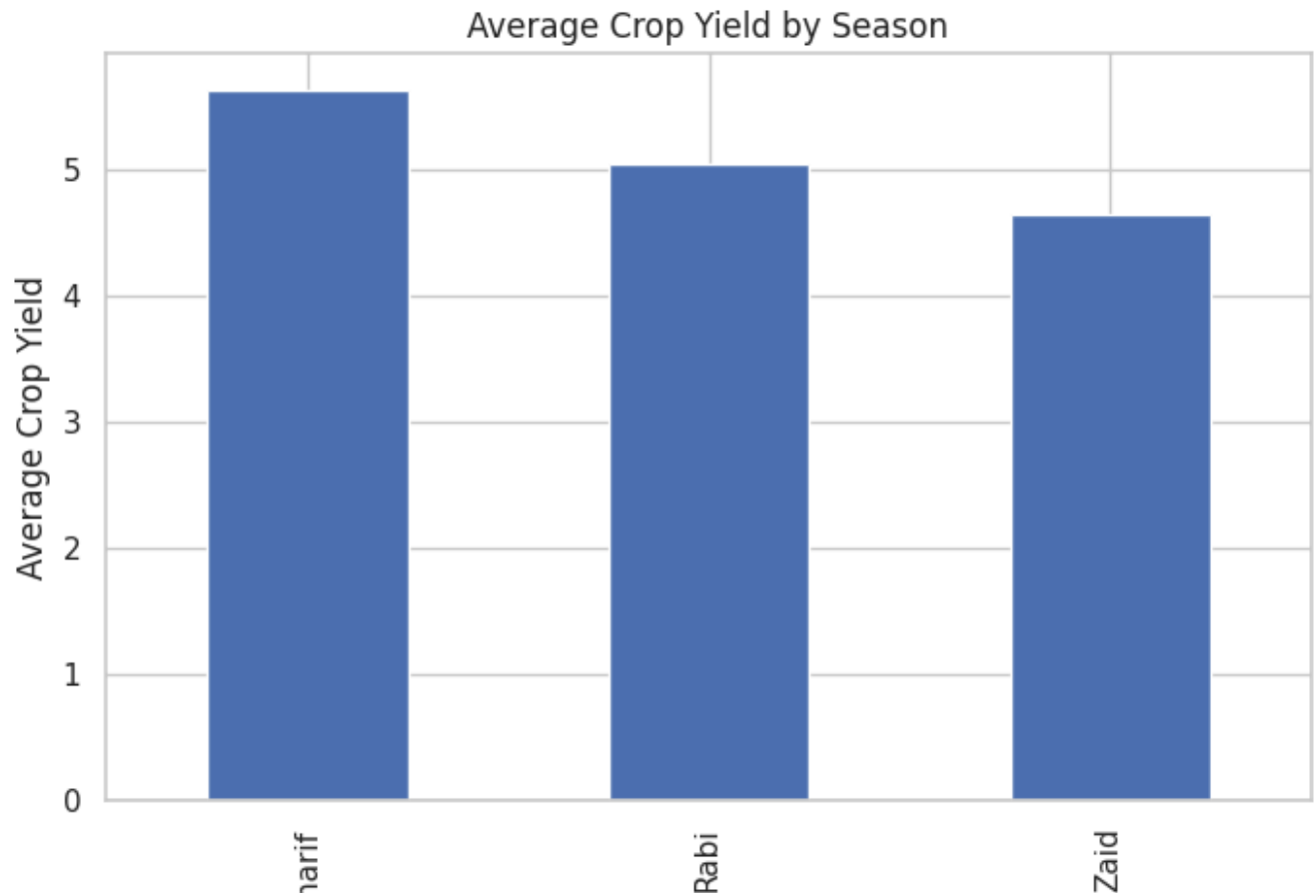

		Average_Yield	Average_Profit	Average_Water_Efficiency
Season	Crop			
Kharif	Sugarcane	53.46	1,000,790.81	36.00
	Maize	2.97	-39,332.67	6.05
	Rice	2.70	-64,903.40	2.17
	Wheat	2.25	-105,871.87	4.76
	Chilli	1.73	954,380.42	3.26
	Groundnut	1.48	108,265.44	3.58
	Cotton	1.37	216,999.55	2.13
	Pulses	1.04	43,338.90	3.35
Rabi	Sugarcane	43.29	731,612.86	27.92
	Maize	2.60	-89,133.40	5.46
	Rice	2.32	-98,427.87	1.84
	Wheat	2.06	-119,954.57	4.69
	Chilli	1.47	638,572.50	2.89
	Groundnut	1.23	10,416.54	2.90
	Cotton	1.20	107,343.59	1.94
	Pulses	0.88	-14,309.14	2.70
Zaid	Sugarcane	38.42	583,635.80	23.39
	Maize	2.29	-194,049.17	4.75
	Rice	1.90	-227,239.34	1.53
	Wheat	1.75	-193,208.95	4.02
	Chilli	1.20	448,659.09	2.20
	Groundnut	1.04	-68,872.50	2.47
	Cotton	0.96	-53,925.34	1.49
	Pulses	0.67	-139,227.81	1.95

```
#analysis 1
#Best Season for Crop Production
season_yield = df.groupby('Season')['Yield_Tonnes_Ha'].mean().sort_values(ascending=False)

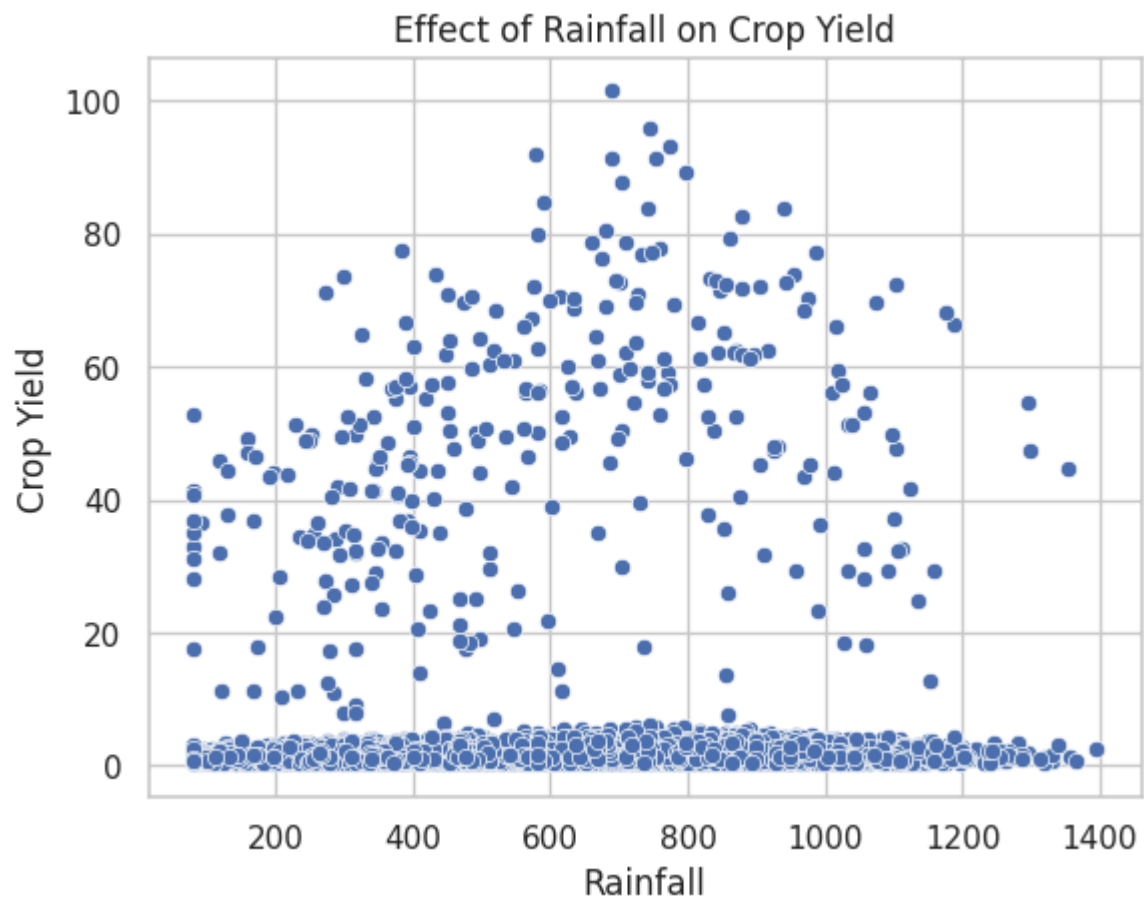
print(season_yield)

season_yield.plot(kind='bar', figsize=(8,5))
plt.title("Average Crop Yield by Season")
plt.ylabel("Average Crop Yield")
plt.show()
```

Season
Kharif 5.63
Rabi 5.04
Zaid 4.64
Name: Yield_Tonnes_Ha, dtype: float64



```
#analysis 2
#Effect of Rainfall on Crop Yield
sns.scatterplot(data=df, x='Rainfall_mm', y='Yield_Tonnes_Ha')
plt.title("Effect of Rainfall on Crop Yield")
plt.xlabel("Rainfall")
plt.ylabel("Crop Yield")
plt.show()
```



```
#analysis 3
#Seasonal Resource Usage
resource = df.groupby('Season')[['Water_Efficiency_t_per_1000m3', 'Fertilizer_kg_ha']]

print(resource)

resource.plot(kind='bar', figsize=(9,5))
plt.title("Resource Usage Across Seasons")
plt.ylabel("Average Usage")
plt.xticks(rotation=45)
plt.show()
```

	Water_Efficiency_t_per_1000m3	Fertilizer_kg_ha
Season		
Kharif	5.89	187.08
Rabi	5.19	185.41
Zaid	4.41	184.64

Resource Usage Across Seasons